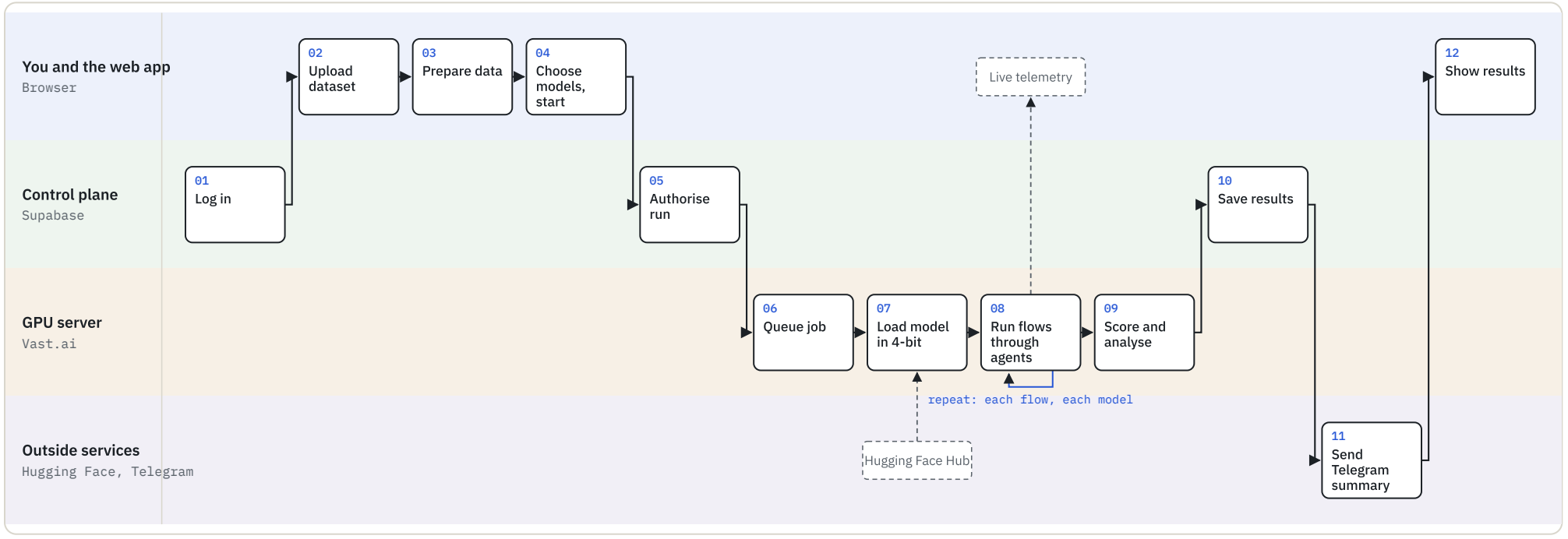
AGENTMETER · SYSTEM FLOW

From login to results, step by step

AgentMeter measures how much **time, GPU memory, tokens and energy** open-source language models use when they run a multi-agent network threat-detection pipeline. It is a benchmarking tool, not a threat detector. This page follows one benchmark session through the whole system.

Web app in the browser · Control plane (Supabase) · Rented GPU server · Outside services



The twelve steps

01 Log in CONTROL PLANE

The SOC Administrator enters a username and password. The control plane checks the password against its stored salted hash and issues an access token. Repeated wrong passwords block login for 15 minutes, and a new user must set their own password on first login. Accounts are created only by an administrator.

Output: `a signed-in session and the dashboard`

02 Upload a dataset WEB APP

The user starts a new benchmark and supplies labelled network traffic: a CSV of network flows, a PCAP packet capture, or an HTTPS link to either. Links to private or local addresses are refused.

Output: `a raw CSV or PCAP file`

03 Prepare the data BROWSER OR GPU SERVER

The file is checked and turned into a small, fair test set. A PCAP is first converted into network flows with 78 standard flow features. The ground-truth label column is found automatically and moved into a separate labels file, so the models never see the answers. Up to 500 representative flows are then sampled using statistical rules only (duration, volume, protocol, port). A SHA-256 hash fingerprints the prepared set.

By default this runs on the user's own computer, inside the browser, using the same Python code as the server. That way the GPU can keep benchmarking another session at the same time.

Output: `features.csv, labels.csv (kept hidden) and manifest.json with the hash`

04 Choose models and start WEB APP

The user picks two to four open-source models from the catalogue and the agent pipeline to use (four agents by default: perceive, reason, decide, act). The system checks each model can be loaded in 4-bit on the available GPU before the run is allowed.

Output: `a benchmark request`

05 Authorise the run CONTROL PLANE

The control plane checks the user is allowed to run a benchmark and has not passed the hourly session limit, then issues a short-lived run token. The GPU server only accepts work that carries a valid token.

Output: `a signed run token`

06 Queue the job GPU SERVER

The request reaches the GPU server through a Cloudflare Tunnel (HTTPS, no open ports). Jobs wait in a first-in, first-out queue, so only one benchmark uses the GPU at a time and measurements are never disturbed by other work.

Output: `a queued, then running, job`

07 Load each model in 4-bit GPU SERVER HUGGING FACE

For each model, a fresh, isolated worker process starts. It downloads the model from the Hugging Face Hub if it is not cached, loads it with 4-bit NF4 quantisation, records its GPU memory, and measures the GPU's idle power so only the model's own energy is counted later.

Output: `a loaded model and its memory and idle-power baseline`

08 Run every flow through the agents GPU SERVER

Each flow passes through the agent pipeline. Perceive describes the traffic, reason interprets it, decide outputs one of five classes, and act gives the response and MITRE ATT&CK technique. Around every single agent call the system records wall time, time to first token, GPU memory and input and output tokens, while GPU power is sampled through NVML.

Results are written to a temporary SQLite file on the server, so an interrupted run can resume. Progress events are written between the timed calls, never inside them, and the web app shows them as live telemetry: download, loading, each flow, GPU use, memory and power.

Output: `per-flow, per-agent measurements for every model`

09 Score and analyse GPU SERVER

Predictions are compared with the hidden labels to give accuracy and output reliability. Each model gets an efficiency-only score using Simple Additive Weighting over latency, GPU memory and tokens, with accuracy kept out of the score, plus a sensitivity check under other weightings. Energy per flow and cost per 1,000 flows are calculated, models are checked against the user's own limits, and Kruskal-Wallis with Dunn's test and a paired bootstrap confirm whether differences and rankings are real. A PDF report is built.

Output: `session results, analyses and the PDF report`

10 Save the results CONTROL PLANE

The GPU server uploads the results to the control plane, which stores the session and scores in PostgreSQL and the files (report, prepared set, full results) in Supabase Storage. From here on the results stay available even after the rented GPU server is switched off.

Output: `a permanent session record`

11 Notify the user TELEGRAM

If the user turned notifications on, the control plane sends one Telegram message with the session status and the more resource-efficient model. The session is marked as notified so the message is never sent twice.

Output: `a Telegram summary`

12 Show the results WEB APP

The user opens the session and reads the results in five tabs: Overview, Reliability, Performance, Agents and Cost. The Agents tab shows which agent uses the most resources. Compare and Leaderboard rank models across sessions run on the same prepared set (same hash), and Trends shows repeated runs over time. The PDF report can be downloaded.

Output: `the most efficient model, the costliest agent, and the evidence behind both`

Three rules that keep the results fair

The models never see the answers

Labels are split off in step 3 and used only for scoring in step 9. No prompt contains a label.

Nothing disturbs a measurement

One job uses the GPU at a time, each model runs in its own process, and telemetry is written outside the timed calls.

Only like is compared with like

Sessions are compared only when their prepared sets have the same hash, so every model faced exactly the same flows.

Where the data lives

PLACE	WHAT IT HOLDS	HOW LONG
Browser	The dataset while it is prepared on the user's computer	Until the prepared set is sent
GPU server (SQLite)	Per-flow and per-agent measurements during a run	Temporary, until results are uploaded

PLACE	WHAT IT HOLDS	HOW LONG
Supabase PostgreSQL	Users, sessions, scores and analyses	Permanent
Supabase Storage	PDF reports, prepared sets, full result files	Permanent